CIS 3300 · OVERVIEW

Generics and Collections

Overview and objectives

- **Course:** CIS 3300, Intermediate Object-Oriented Programming (Java)
- **Text:** *Core Java, Volume I* (13th ed.), Chapters 8 and 9
- **Materials:** two parts of class notes, with every example tested, and the Speedrun Leaderboard guides for Checkpoints 3 and 4
- **Prerequisite you already have:** arrays and `ArrayList`; classes, records, and interfaces; and inheritance and polymorphism

1. The short version

You have used `ArrayList<String>` since the first week. This unit explains what the angle brackets mean. *Generics* let a class or method work with many types while the compiler still checks every use, so type mistakes are found before a program runs instead of while it runs. You will write generic classes and methods of your own, and learn how bounds and wildcards say exactly which types a method accepts.

The second half is about *collections*: lists, sets, maps, and queues. Each is built for a different job, and the main skill is choosing the right one. You will also learn the two rules that the most common collections depend on, `equals` with `hashCode` for hash collections and a natural order for tree collections, and the mistakes behind the exceptions collections throw most often.

2. What We Will Cover

- **Generics.** Why generics exist and why raw types should not be used, type arguments and the diamond, wrapper classes and autoboxing, generic classes and records, generic methods, bounded type parameters, the wildcards `? extends` and `? super`, and what type erasure prevents.
- **Collections.** The `List`, `Set`, `Map`, `Queue`, and `Deque` interfaces and their common classes; the order of hash, linked, and tree collections; `equals` and `hashCode`; natural order with `Comparable`; removing safely while looping; collections that cannot be changed; and choosing a collection for a task.

3. Objectives

By the end of this unit, you should be able to:

1. Explain the problem generics solve, and why raw types should not be used.
2. Use generic types with type arguments and the diamond, and use wrapper classes and autoboxing to store primitive values, including the risk of unboxing `null`.
3. Write a generic class or record with one or more type parameters.

4. Write a generic method, and explain how the compiler infers its type argument.
5. Use a bounded type parameter, such as `T extends Comparable<? super T>`, and explain what the bound allows the method to do.
6. Explain why a `List<Integer>` is not a `List<Number>`, and choose between `? extends` and `? super` for a method parameter.
7. Describe what type erasure prevents: primitive type arguments, `new T()`, and `instanceof` with a type argument.
8. Describe the `List`, `Set`, `Map`, and `Queue` interfaces, and choose an appropriate class for a given task.
9. Predict the order in which hash, linked, and tree collections return their elements.
10. Override `equals` and `hashCode` together, and explain what goes wrong in a hash collection when only `equals` is overridden.
11. Give a class a natural order with `Comparable`, so it can be sorted and kept in a tree collection.
12. Remove elements from a collection safely while looping over it.
13. Create collections that cannot be changed, tell a copy from a view, and return copies from getters to protect encapsulation.
14. Use an `ArrayDeque` as a queue and as a stack.

4. How to Show Up Ready

- **Read ahead.** Skim Core Java, Volume I, Chapter 8 for generics and Chapter 9 for collections.
- **Look back at your own code.** Find every collection you have used in your labs, and ask whether a different kind would have done the job better.
- **Review interfaces and `Comparable`.** Collections are organized as interfaces, and the tree collections depend on `compareTo`.
- **Run the examples yourself.** Type them in, change them, and predict the output before you run them. Several of them fail on purpose; read the error message before you fix it.
- **If you miss a class,** work through that day's Speedrun Leaderboard checkpoint guide from its starting project before the next class.